

Circle x Ignyte Submission — DeFa on Arc

This document maps 1:1 to the Ignyte submission form. Each section corresponds to one form field so it can be copied straight in.

1. Title and Short Description

Title: DeFa — On-chain credit operating infrastructure for SME trade finance, settling in USDC on Arc.

Short description: DeFa is a factory-cloned credit-pool protocol built for cross-border SME trade finance. Liquidity Providers deposit USDC into risk-tiered pools; Payment Service Providers and SMEs draw against verified receivables; KAM → CAD → CRO → Legal approvals gate every facility before an on-chain admin signs the pool bootstrap. Repayment waterfalls are automated on-chain; defaults settle from a treasury reserve. Settlement is denominated in USDC end-to-end, running today on Arc Testnet.

2. Track submitted for

- **Primary — Track 2: SME Trade Finance & Working Capital**
 - **Secondary — Track 4: Agentic Economy** (*the KAM → CAD → CRO → Legal approval chain is agent-ready — each seat's decision is bounded by structured inputs and can be run by a deterministic agent tomorrow*)
-

3. Email associated with your Circle Developer Account

hamzaanjum013@gmail.com

Registered via <https://console.circle.com/signup>. Used to obtain testnet USDC via the Circle Public Faucet.

4. Circle products used on Arc

- ☒ USDC
- ☐ Wallets
- ☐ Gateway
- ☐ CCTP / Bridge Kit
- ☐ USYC
- ☐ StableFX
- ☐ Nanopayments

How USDC is used in DeFa today (Arc Testnet):

- Every pool is denominated in USDC. `PoolContract.deposit(amount)` , `PoolContract.executeDrawdown(...)` , and `PoolContract.repay(...)` all move USDC.
- Testnet USDC was sourced via the [Circle Public Faucet](#) to fund the deployer wallet, agent wallet, and demo LP/PSP accounts on Arc Testnet.
- The `payfi_v1` contract set uses 6-decimal WAD math throughout, so the same code drops onto mainnet-USDC on Arc mainnet without a decimal shim — this is a direct dividend of Arc's stablecoin-native design.

Circle products explicitly on our roadmap (Phase 2 — Q4 2026):

- **CCTP** — cross-chain drawdowns: LP funds pool on Arc, PSP draws USDC on Ethereum L1 supplier wallet, waterfall repay routes back to Arc. Our backend already routes through structured `build-tx` endpoints so adding a CCTP-message step is a targeted addition, not a rewrite.
- **Circle Wallets (Programmable Wallets)** — Phase 3 (H1 2027). Optional embedded-wallet onboarding for SMEs that don't want to manage MetaMask themselves, alongside our current SIWE + external-wallet flow.
- **Gas Station** — Phase 2. Sponsored gas for SME PSPs so they never need to hold USDC just to sign a drawdown.

5. Functional MVP and architecture diagram

Working frontend + backend + smart contracts, all live on Arc Testnet:

Layer	Status	Where
Smart contracts (payfi_v1)	✓ Deployed	Arc Testnet (chain 5042002) — PoolFactory , MockUSD , TreasuryReserve
Backend	✓ Deployed	https://defa-arc-hackathon.invoicemate.net/api (Node/Express/Mongoose/ethers v6)
Frontend — Lender UI	✓ Deployed	https://defa-arc-hackathon.invoicemate.net/ (React 19 + wagmi + RainbowKit)
Frontend — PSP + Admin UI	✓ Deployed	https://defa-arc-hackathon-admin.invoicemate.net/

Architecture diagram: [docs/ARCHITECTURE.md](#) — three Mermaid diagrams (top-level system, facility lifecycle sequence, contract class diagram).

Test coverage: 38 integration tests across `e2eFlows` + `lifecycleFlows` + `onchainFlows` suites, running green on real Arc Testnet ([server/TESTING.md](#)). Foundry contract suite covers ~17K LOC across 26 test files (unit, invariant, adversarial, differential, fuzz, gas-profile).

6. Video demonstration + presentation

Demo video: `<paste YouTube unlisted link when uploaded>`

Presentation deck: `<paste Google Slides / PDF link when uploaded>`

Video outline (2 minutes, per the shot list in the repo):

- 0:00 — Problem: cross-border payment providers lock millions in pre-funding accounts.
- 0:07 — Solution: DeFa on-chain liquidity infrastructure on Arc.
- 0:18 — 28-item onboarding + AI KYR scoring (79/100 rating shown).
- 0:37 — Facility created on-chain by admin, visible in on-chain-admin dashboard.
- 0:49 — Lender browses live facilities on Arc.
- 1:02 — LP deposits USDC into a facility. MetaMask signs approve + deposit. Real onchain txs.
- 1:11 — PSP initiates drawdown against a real external customer order.
- 1:26 — Validation pipeline runs 5 checks. All pass; liquidity transfers.
- 1:39 — Daily activity + P&L breakdown surfaces on the borrower + lender dashboards.
- 1:46 — Credit memo / KYR report available to LPs as a downloadable PDF.

- 2:08 — DeFa is now live on Arc Testnet.

7. Link to GitHub / Code repository

Repository: <https://github.com/Mate-Sol/The-Stablecoins-Commerce-Stack-Challenge-Arc-Circle->

Setup instructions: [docs/LOCAL_E2E.md](#) — full local dev walkthrough (Anvil + Foundry + Mongo + server + client).

Circle integration reference: [docs/CONTRACTS.md](#) surfaces where USDC transfers happen in the contract layer. [docs/API.md](#) documents the backend `/pool/lender/build-tx/deposit` endpoint that assembles the USDC approve + deposit calldata for LPs.

Additional docs in the repo:

- [README.md](#) — landing page + how-to-test with credentials for all 6 roles
- [docs/ARCHITECTURE.md](#) — Mermaid architecture diagrams
- [docs/SUBMISSION.md](#) — long-form submission narrative
- [docs/ROADMAP.md](#) — 5-phase launch roadmap
- [SECURITY.md](#) — contract security posture + audit plan

8. Demo Application Platform / Application URL

- **Lender / LP UI:** <https://defa-arc-hackathon.invoicemate.net/>
- **PSP + Admin UI:** <https://defa-arc-hackathon-admin.invoicemate.net/>
- **Onchain Admin flow:** <https://defa-arc-hackathon-admin.invoicemate.net/onchain-admin/login>

Access code for LP signup on the live demo: 654321

Test wallet (pre-funded on Arc Testnet with 1M MockUSD + on the on-chain admin allowlist):

```
Private key: 0x692fb6f9b2c22e3d2ad4e0434f22f41617fb65ce1ac89146da2ae21b58443ce9
Address:    0x0b9dDfcdB31aEf5Cde26d0E6DbAc6917B6849f05
```

(Deliberately published for the hackathon — will be rotated post-submission. Treated as a burner wallet and pre-authorised in `.env.production.example`.)

9. Circle Product Feedback

Why we chose these products for our use case

- **USDC** was the only viable settlement asset for our use case. Cross-border SME trade finance is priced in USD but drawn and settled in local currencies; USDC lets us denominate the credit in USD while keeping settlement instantaneous and on-chain.
- **Circle Public Faucet** was mandatory infrastructure — we needed testnet USDC on the deployer wallet, the AGENT signer, and every LP/PSP account we used in demos. Without it, we couldn't run the 38-test integration suite against real Arc Testnet state.
- **Arc as the chain** was picked because USDC is the gas asset. In our lender UX we dropped an entire "get testnet ETH from a faucet before you can deposit" wizard — users just need one asset. On other EVM chains we've had to write per-asset decimal shims for 6-decimal USDC math; on Arc it's the same asset the pool settles in.

What worked well during development

- **USDC-as-gas is the right primitive.** Removing the "user needs a separate gas asset" step is worth an entire onboarding-flow simplification. In our lender UX, we dropped the "get testnet ETH from a faucet before you can deposit" wizard entirely.
- **6-decimal-native math.** Our `payfi_v1` contract set uses 6-decimal WAD math throughout. On other EVM chains we've had to write per-asset decimal normalisers; on Arc it just drops in.
- **Testnet RPC uptime.** Solid enough to run the full 38-test integration suite against real Arc Testnet multiple times per day without RPC being the flaky link.
- **Block explorer.** `testnet.arcscan.app` was fast enough that we could verify state changes within a block of confirmation — critical for a live-demo debugging loop.

What could be improved

- **Faucet rate limiting was aggressive at the start of the sprint.** Anti-sybil is important, but the "you need mainnet history to claim testnet USDC" gate on the primary faucet became a real blocker for hackathon teams onboarding fresh wallets. We eventually funded from another wallet with mainnet history, but the first two days of our build were faucet-hunting instead of shipping. A dedicated hackathon faucet with a code-based unlock would be a huge quality-of-life win.
- **Documentation on chain-id + native-currency-config for MetaMask** was thin. We had to trace the values from block-explorer metadata. A canonical "Add Arc Testnet to MetaMask" one-click button on `docs.arc.build` would save every team the same 20 minutes.
- **Testnet USDC contract discovery** was harder than it needed to be. We shipped a `MockStablecoin` clone because we couldn't find an authoritative "canonical testnet USDC address" in the docs during the sprint window. Publishing a canonical testnet USDC address in `docs.arc.build` next to the RPC endpoints would remove this ambiguity.
- **CCTP on testnet is not obvious to discover.** We wanted to demo cross-chain drawdown but ran out of time locating the CCTP testnet contract map for Arc. If it exists it should be linked from the chain-info page; if it doesn't exist yet, a target date would help teams roadmap around it.

Recommendations to make the product / developer experience more seamless or scalable

1. **Native KYB attestation at the ERC-20 layer.** For regulated PSPs, the ability to declare "this address is KYB'd" as a first-class contract attestation would remove an entire allowlist-proxy pattern we currently deploy on-chain. Circle is uniquely positioned to issue this given its compliance surface.
2. **A canonical "receivables / trade finance" reference implementation in `arc-examples`.** Trade finance is one of the largest greenfield use cases for programmable stablecoins, but the current examples set is heavy on payments and light on credit primitives. We would contribute our `payfi_v1` set as a starting point.
3. **CCTP-native cross-chain USDC drawdowns from a single pool.** LP funds on Arc, PSP draws to Ethereum L1 supplier wallet, waterfall repay flows back to Arc. If Circle can package this as a single Arc-side contract entry point, it's an enormous unlock for cross-border credit — every trade-finance corridor we operate is inherently multi-chain.
4. **A Circle-branded compliance oracle** that pool admins can reference during pool bootstrap ("only allow drawdowns to Circle-attested KYB'd receivers"). Regulatory teams at partner banks would find this materially easier to sign off on than an in-house allowlist.
5. **A dedicated Arc hackathon faucet** with a code-based unlock (or team-badge whitelist) so future hackathon cohorts don't lose their first day to faucet hunting.